

事件（消息）驱动的程序设计

一.Windows与事件驱动的程序设计

1.Windows简介

1. Windows是一种基于图形界面的多任务操作系统。
 - 系统中可以同时运行多个程序
 - 每个程序通过各自的窗口与用户进行互动
 - 用户通过鼠标的单击/双击/拖动、菜单的选择以及键盘输入来与程序进行交互
2. Windows的功能以两种方式提供：
 - 工具（程序）：资源管理器、记事本、画图、.....
 - 函数库：以C语言函数形式出现（在windows.h等头文件中申明），作为应用程序接口（API），供Windows应用程序使用

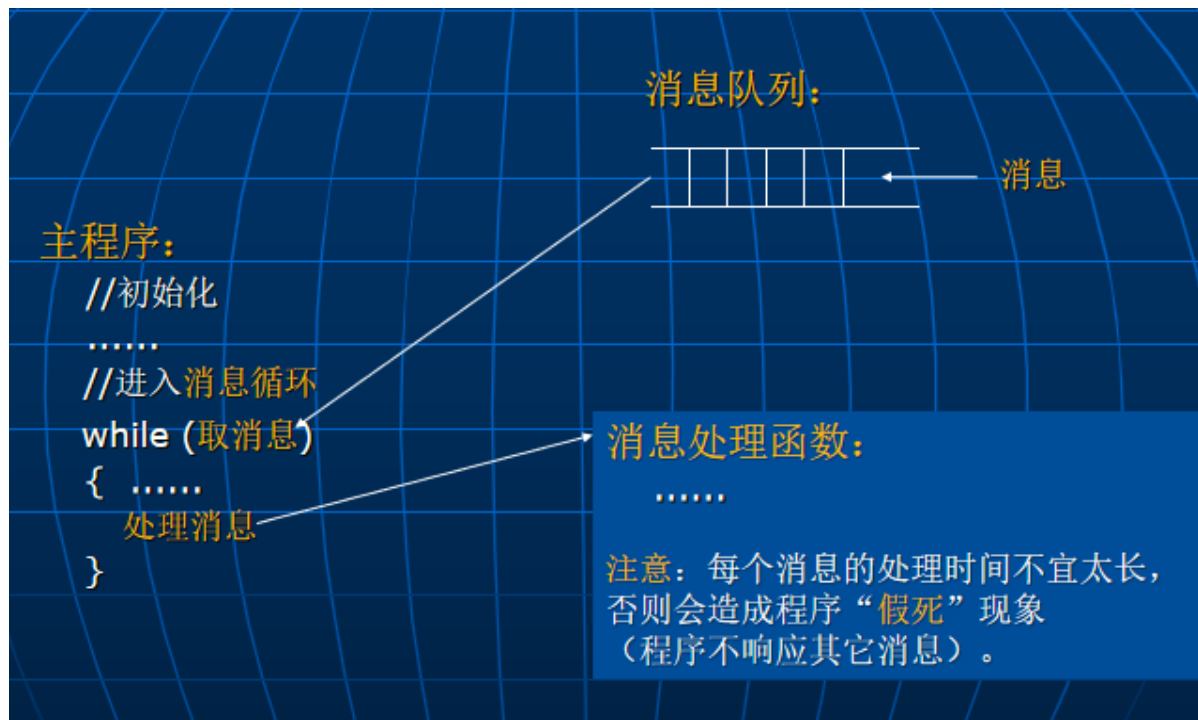
2.Windows应用程序的类型

1. 单文档应用：
 - 只能对一个文档的数据进行操作
 - 必须首先等当前文档的所有操作结束后，才能进行下一个文档的操作
2. 多文档应用：
 - 同时对多个文档的数据操作
 - 不必等到一个文档的所有操作结束，就可以对其他文档进行操作，对不同文档的操作是在不同的子窗口中进行的
3. 对话框应用：
 - 以对话框的形式操作一个文档数据
 - 对文档数据的操作以各种“控件”来实现
 - 程序以按<确定>或<取消>等按钮来结束

3.事件驱动的程序结构

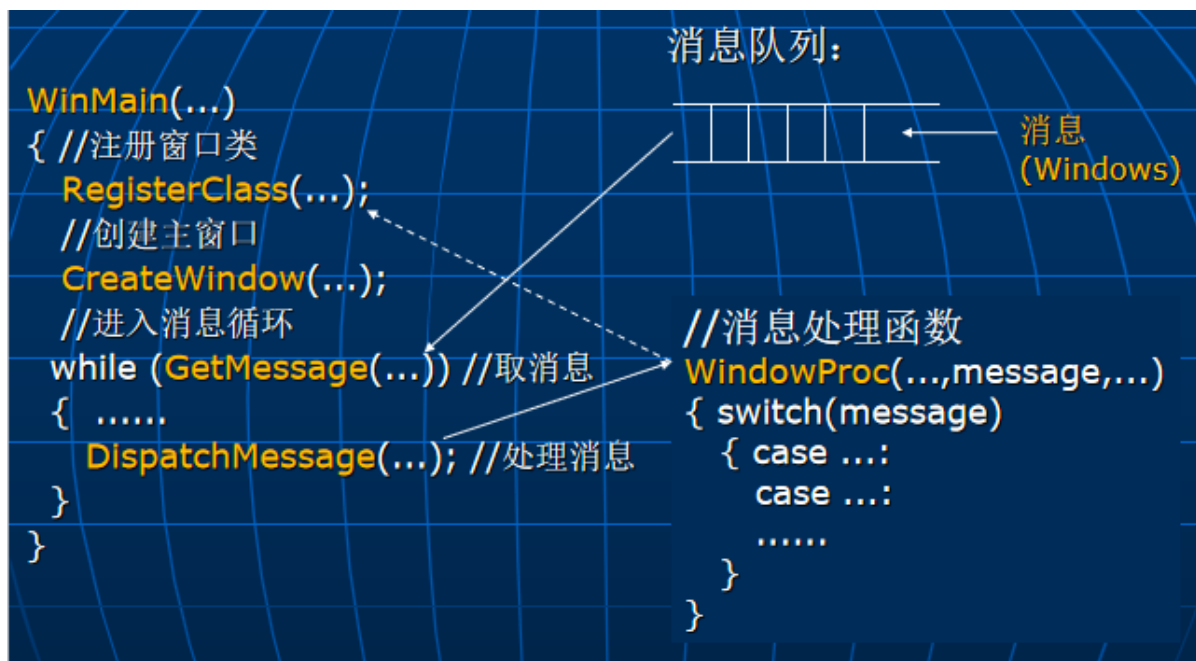
1. Windows应用程序采用的是一种事件驱动的交互式流程控制结构：
 - 程序的任何一个动作都是由某一个事件激发的
 - 事件可以是用户的键盘、鼠标、菜单等操作
2. 每个事件都会向应用程序发送一些消息，如：
 - WM_KEYDOWN/WM_KEYUP（键盘按键）
 - WM_CHAR（按键有对应的字符）
 - WM_LBUTTONDOWN/WM_LBUTTONUP/WM_LBUTTONDBLCLK/WM_MOUSEMOVE（鼠标按键）
 - WM_COMMAND（菜单选取）
 - WM_PAINT（窗口内容刷新）
 - WM_TIMER（定时器时间到了）
3. 每个应用程序都有一个消息队列

- Windows系统会把属于各个应用程序的消息放入各自的消息队列中
 - 应用程序不断地从自己的消息队列中获取消息并处理之
 - “取消息-处理消息”的过程被成为消息循环
 - 当取到某个特定消息后，消息循环结束
4. 每个窗口都有一个消息处理函数
- 大部分的消息都关联到某个窗口
 - 应用程序取到消息后将会去调用相应窗口的消息处理函数



二.基于Windows API的过程式的事件驱动程序设计

1. 每个Windows应用程序都必须提供一个主函数WinMain,其主要功能是：
- 注册窗口类（定义程序中要创建的窗口类型）：
 - 窗口的基本风格、消息处理函数、图标、光标、背景颜色以及菜单等
 - 每类窗口（不是每个）都需要注册
 - 根据注册的窗口类创建应用程序的**主窗口**（程序的其他窗口等到需要时再创建）
 - 进入**消息循环**，直到接受到WM_QUIT消息时，消息循环结束



```
1 //windows应用程序的主函数
2 #include <windows.h> //Windows所提供的API声明文件。
3 int APIENTRY WinMain(HINSTANCE hInstance, //本实例标识
4                     HINSTANCE hPrevInstance, //上一个实例标识
5                     LPSTR lpCmdLine, //命令行参数
6                     int nCmdShow ) //主窗口的初始显示
7                     方式
8 { //注册窗口类（下面是个示意，函数参数实际为一个结构WNDCLASS）
9   RegisterClass(..., WindowProc//消息处理函数," my_window_class"//窗口类的名字); //示意
10   .....
11 //创建并显示主窗口
12 HWND hwnd; //窗口的id
13 hwnd=CreateWindow("my_window_class",...,x,y,width,height,);
14 ShowWindow(hwnd, nCmdShow);
15 .....
16 //消息循环，直到接收到WM_QUIT消息
17 while (GetMessage(&msg, NULL, 0, 0)) //从消息队列中取消息。
18 { .....
19   DispatchMessage(&msg); //把消息发送到程序相应的窗口。
20 }
21 return msg.wParam;
22 //窗口的消息处理函数
23 LRESULT CALLBACK windowProc(HWND hwnd, //窗口标识
24                             UINT message, //消息标识
25                             WPARAM wParam, //消息的参数1
26                             LPARAM lParam ) //消息的参数2
27 { switch (message)
28   { case WM_KEYDOWN:
29     ...wParam... //wParam为按键的名字
30     .....
31     case WM_CHAR: //字符键消息
32     ...wParam... //wParam是按键对应字符的编码
33     .....
34     case WM_COMMAND: //菜单消息
35     switch (wParam) //wParam是菜单项的标识
```

```

36         { case ID_FILE_OPEN: //打开文件
37             .....
38             case ID_START_TIMER: //启动定时器
39                 SetTimer(hwnd,1,1000,NULL);
40         }
41         .....
42     case WM_LBUTTONDOWN: //鼠标左键按下消息
43         ...lParam... //lParam为鼠标在窗口中的位置
44         .....
45     case WM_PAINT: //窗口刷新消息
46         .....
47     case WM_TIMER: //定时器消息
48         ...wParam... //wParam是定时器编号
49         .....
50     case WM_CLOSE: //关闭窗口消息
51         DestroyWindow(hwnd); //撤销窗口
52         break;
53     case WM_DESTROY: //窗口被撤销消息
54         PostQuitMessage(0); //往本应用的消息队列中放入WM_QUIT
55         break;
56     default: //由系统进行默认的消息处理
57         return DefWindowProc(hwnd,message,wParam, lParam);
58     }
59     return 0;
60 }
61

```

2. 资源：

- 每个Windows应用程序，除了程序代码外，还包含一些资源：
 - 菜单：菜单ID、菜单项ID/显示文字
 - 对话框：对话框ID和尺寸；对话框中各个控件的ID、类型、尺寸和位置等
- 资源的描述有规定的格式，存储在相应的资源文件（.rc）中
- 资源可以用VC++的资源编辑器来进行可视化编辑

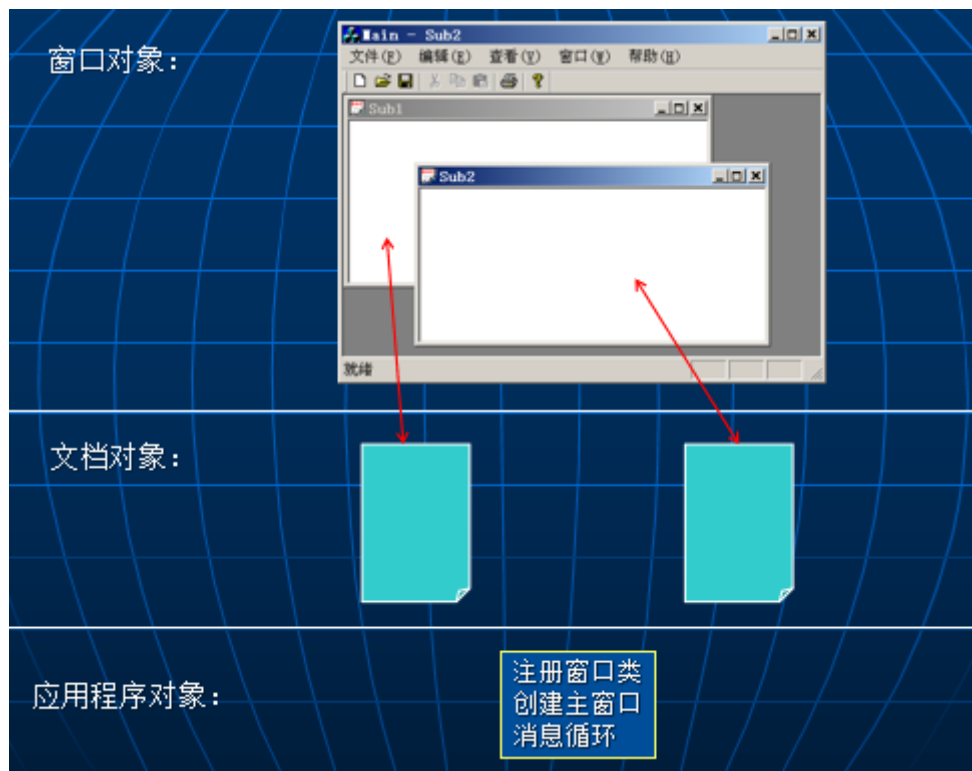
三.面向对象的事件驱动程序设计

基于Windows API的事件驱动程序设计属于过程式程序设计范式。通过调用API函数编写程序的粒度太细、太繁琐，开发效率不高。如何以更大粒度的程序元素（如，类/对象）来开发事件驱动的应用程序？

1. Windows应用程序中的对象：

- 窗口对象：
 - 显示程序的处理数据
 - 处理Windows的消息、实现与用户的交互
 - 窗口对象类之间可以存在继承和组合关系
- 文档对象：
 - 管理在各个窗口中显示和处理的数据
 - 文档对象与窗口对象之间可以存在关联关系
- 应用程序对象：

- 管理属于它的窗口对象和文档对象
- 实现消息循环
- 它与窗口对象及文档对象之间构成了组合关系



2. 支持Windows应用开发的类库

- 微软的MFC，用类封装了Windows API的功能
- Qt公司的Qt:不需要了解太多的底层实现细节，跨平台

3. 应用程序可以通过继承上面类库中的类来实现各自的特殊功能

四.基于应用框架的事件驱动程序设计

应用框架是一种通用的、可复用的应用程序结构，该结构规定了程序应包含哪些组件以及这些组件之间的关系，它封装了程序处理流程的控制逻辑。

通过复用应用框架，使得应用的开发速度更快、质量更高、成本更低。

- 在一个应用框架中，各个组件以及它们之间的关系是固定的，应用的开发者通过给各组件添加具体的业务代码来实现不同的应用。
- 应用框架有很多，例如：基于MFC的“文档-视”结构的应用框架;基于Qt的“模型-视”结构的应用框架